MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 6: SMART SURVEILLANCE CAMERA USING ESP32-CAM

SECTION 1

GROUP 9

INSTRUCTOR: DR WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV.

DR ZULKIFLI BIN ZAINAL ABIDIN

NO	NAME	MATRIC NO
1	NOR EFFENDI BIN ANUAR @ ANWAR	2310519
2	NUR AINA SAFFIYA BINTI MOHD TARMIZI	2411644
3	NUR ATIKAH BINTI KAMARUL BAHARIN	2319846

DATE OF SUBMISSION: 15 APRIL 2026

TABLE OF CONTENTS

1.0 INTRODUCTION

This report explores the integration of Internet of Things (IoT) and mechatronic actuators through the development of a smart surveillance system using the ESP32-CAM. The experiment focuses on implementing live video streaming over Wi-Fi and utilizing Pulse Width Modulation (PWM) to control a servo motor for horizontal panning, simulating active motion tracking. By constructing this prototype, core mechatronic concepts are demonstrated, including sensor fusion and real-time hardware response, effectively bridging the gap between digital signal processing and physical system automation in a security context. The laboratory tasks specifically address manual control through a pushbutton and the implementation of advanced face detection capabilities to guide motor movement.

2.0 MATERIALS AND EQUIPMENT

2.1 Task 1

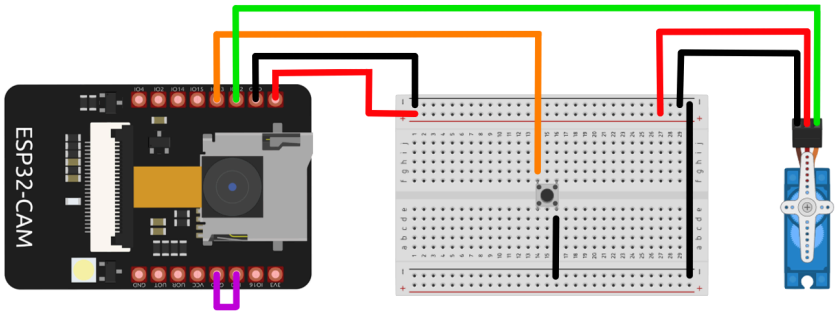
- Microcontroller: ESP32-CAM
- Input component: Push button
- Output component: Servo motor
- Prototyping: Breadboard and Jumper wires
- Software: Arduino IDE
- Libraries:
    - Arduino: Servo.h, WiFi.h, esp_camera.h

2.2 Task 2

- Microcontroller: ESP32-CAM
- Input component: EP32-CAM camera
- Output component: Servo motor
- Prototyping: Breadboard and Jumper wires
- Software: Arduino IDE
- Libraries:
    - Arduino: Servo.h, WiFi.h, esp_camera.h, camera_pins.h

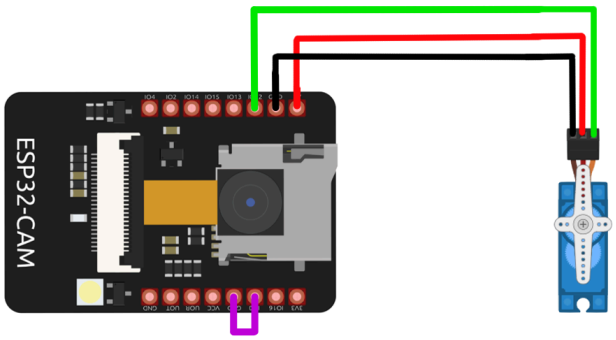
3.0 EXPERIMENTAL SETUP

3.1 Task 1



1. Connect servo motor to ESP32-CAM:
    - Connect the red wire of the servo motor to the 5V terminal on the breadboard.
    - Connect the orange wire to the GPIO12 on ESP32-CAM.
    - Connect the black wire of the servo motor to the GND terminal on the breadboard.
2. Connect push button to ESP32-CAM:
    - Connect one side of the push button to GPIO13 on ESP32-CAM.
    - Connect the opposite side of the push button to the GND terminal on the breadboard.

3.2 Task 2



3. Connect servo motor to ESP32-CAM:
    - Connect the red wire of the servo motor to the 5V terminal on the breadboard.
    - Connect the orange wire to the GPIO12 on ESP32-CAM.
    - Connect the black wire of the servo motor to the GND terminal on the breadboard.

4.0 METHODOLOGY

4.1 Task 1

4.1.1 Circuit Setup

- Interface the ESP32-CAM with the CH340 USB to TTL Serial Cable, ensuring the 5V, GND, U0R, and U0T pins are correctly mapped to provide power and communication.
- Connect the servo motor to the designated PWM-capable GPIO pin on the ESP32-CAM.
- Connect one leg of a physical pushbutton to GPIO 13 and the other leg to GND to serve as the manual trigger.
- Ensure the servo is powered by an external 5V 2A source rather than the USB cable alone to prevent voltage drops or board damage.

4.1.2 Program Development and Uploading

- Bridge the IO0 pin to GND to enter flash mode and select "ESP32 WROVER MODULE" with the "Huge APP" partition scheme in the Arduino IDE.
- Edit the CameraWebServer example to include a conditional if statement that checks the digital state of GPIO 13.
- Define GPIO 13 as an INPUT_PULLUP in the software to use the internal resistor for signal stability.
- Insert the local Wi-Fi SSID and password, then compile and upload the sketch to the module.

4.1.3 Program Operation

- Disconnect the IO0-to-GND bridge and press the reset (RST) button to start the program.
- Open the Serial Monitor to obtain the IP address and access the web portal, verify that the servo remains stationary until the physical pushbutton is held down.

4.1.4 Coding

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESP32Servo.h>
#include <ezButton.h>

//
// WARNING!!! PSRAM IC required for UXGA resolution and high JPEG
quality
// Ensure ESP32 Wrover Module or other board with PSRAM is selected
// Partial images will be transmitted if image exceeds buffer size
// You must select partition scheme from the board menu that has at
least 3MB APP space.
// Face Recognition is DISABLED for ESP32 and ESP32-S2, because it
takes up from 15 seconds to process single frame. Face Detection is
ENABLED if PSRAM is enabled as well

// ===================
// Select camera model
// ===================
//#define CAMERA_MODEL_WROVER_KIT // Has PSRAM
//#define CAMERA_MODEL_ESP_EYE // Has PSRAM
//#define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
//#define CAMERA_MODEL_M5STACK_PSRAM // Has PSRAM
//#define CAMERA_MODEL_M5STACK_V2_PSRAM // M5Camera version B Has
PSRAM
//#define CAMERA_MODEL_M5STACK_WIDE // Has PSRAM
//#define CAMERA_MODEL_M5STACK_ESP32CAM // No PSRAM
//#define CAMERA_MODEL_M5STACK_UNITCAM // No PSRAM
#define CAMERA_MODEL_AI_THINKER // Has PSRAM
//#define CAMERA_MODEL_TTGO_T_JOURNAL // No PSRAM
//#define CAMERA_MODEL_XIAO_ESP32S3 // Has PSRAM
// ** Espressif Internal Boards **
//#define CAMERA_MODEL_ESP32_CAM_BOARD
//#define CAMERA_MODEL_ESP32S2_CAM_BOARD
//#define CAMERA_MODEL_ESP32S3_CAM_LCD
//#define CAMERA_MODEL_DFRobot_FireBeetle2_ESP32S3 // Has PSRAM
//#define CAMERA_MODEL_DFRobot_Romeo_ESP32S3 // Has PSRAM
```

```
#include "camera_pins.h"

// ===========================
// Enter your WiFi credentials
// ===========================
const char* ssid = "pendinyer";
const char* password = "fendibcv";

void startCameraServer();
void setupLedFlash(int pin);

const int servoPin = 14;
const int buttonPin = 13;

Servo myservo;
ezButton mybutton(buttonPin);

int currentAngle = 0;

void setup() {

  myservo.attach(servoPin);
  myservo.write(currentAngle);

  mybutton.setDebounceTime(50);

  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Serial.println();

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.frame_size = FRAMESIZE_UXGA;
  config.pixel_format = PIXFORMAT_JPEG; // for streaming
```

```
  //config.pixel_format = PIXFORMAT_RGB565; // for face
detection/recognition
  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
  config.fb_location = CAMERA_FB_IN_PSRAM;
  config.jpeg_quality = 12;
  config.fb_count = 1;

  // if PSRAM IC present, init with UXGA resolution and higher JPEG
quality
  //                      for larger pre-allocated frame buffer.
  if(config.pixel_format == PIXFORMAT_JPEG){
    if(psramFound()){
      config.jpeg_quality = 10;
      config.fb_count = 2;
      config.grab_mode = CAMERA_GRAB_LATEST;
    } else {
      // Limit the frame size when PSRAM is not available
      config.frame_size = FRAMESIZE_SVGA;
      config.fb_location = CAMERA_FB_IN_DRAM;
    }
  } else {
    // Best option for face detection/recognition
    config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
    config.fb_count = 2;
#endif
  }

#if defined(CAMERA_MODEL_ESP_EYE)
  pinMode(13, INPUT_PULLUP);
  pinMode(14, INPUT_PULLUP);
#endif

  // camera init
  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }

  sensor_t * s = esp_camera_sensor_get();
  // initial sensors are flipped vertically and colors are a bit
saturated
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); // flip it back
    s->set_brightness(s, 1); // up the brightness just a bit
    s->set_saturation(s, -2); // lower the saturation
  }
  // drop down frame size for higher initial frame rate
  if(config.pixel_format == PIXFORMAT_JPEG){
    s->set_framesize(s, FRAMESIZE_QVGA);
  }
```

```
#if defined(CAMERA_MODEL_M5STACK_WIDE) ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)
  s->set_vflip(s, 1);
  s->set_hmirror(s, 1);
#endif

#if defined(CAMERA_MODEL_ESP32S3_EYE)
  s->set_vflip(s, 1);
#endif

// Setup LED FLash if LED pin is defined in camera_pins.h
#if defined(LED_GPIO_NUM)
  setupLedFlash(LED_GPIO_NUM);
#endif

  WiFi.begin(ssid, password);
  WiFi.setSleep(false);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  startCameraServer();

  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  Serial.println("' to connect");
}

void loop() {
mybutton.loop();


  if (mybutton.getState() == LOW) {
    if (currentAngle < 180) {
      currentAngle += 1;
      myservo.write(currentAngle);
      Serial.println(currentAngle);
      delay(10);
    }
  }

  else {
    if (currentAngle > 0) {
     currentAngle = 0;
      myservo.write(currentAngle);
      Serial.println(currentAngle);
    }
  }
}
```

4.2 Task 2

4.2.1 Circuit Setup

- Maintain the existing connections between the ESP32-CAM and the servo motor, ensuring the camera lens has an unobstructed field of view for panning.
- Secure the ESP32-CAM onto the rotating servo base to ensure that mechanical movement translates into a corresponding shift in the video stream's perspective.

4.2.2 Program Development and Uploading

- Modify the camera server code to utilize the built-in face detection capabilities of the OV2640 module.
- Implement logic that reads the X-coordinates of a detected face from the web interface.
- Assign specific PWM commands to the servo: if the face coordinate is on the "Left," the servo rotates left; if "Right," it rotates right; and if "Center," it holds its position.

4.2.3 Program Operation

- Access the camera's web server through a browser and navigate to the "Face Detection" toggle to activate the vision processing stream.
- Observe the system as it identifies a subject's face, the servo should autonomously pan the camera to keep the subject centered based on the mapping data.

4.2.4 Coding

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESP32Servo.h>
#include <ezButton.h>

//
// WARNING!!! PSRAM IC required for UXGA resolution and high JPEG
quality
// Ensure ESP32 Wrover Module or other board with PSRAM is selected
// Partial images will be transmitted if image exceeds buffer size
// You must select partition scheme from the board menu that has at
least 3MB APP space.
// Face Recognition is DISABLED for ESP32 and ESP32-S2, because it
takes up from 15 seconds to process single frame. Face Detection is
ENABLED if PSRAM is enabled as well

// ===================
// Select camera model
// ===================
//#define CAMERA_MODEL_WROVER_KIT // Has PSRAM
//#define CAMERA_MODEL_ESP_EYE // Has PSRAM
//#define CAMERA_MODEL_ESP32S3_EYE // Has PSRAM
//#define CAMERA_MODEL_M5STACK_PSRAM // Has PSRAM
//#define CAMERA_MODEL_M5STACK_V2_PSRAM // M5Camera version B Has
PSRAM
//#define CAMERA_MODEL_M5STACK_WIDE // Has PSRAM
//#define CAMERA_MODEL_M5STACK_ESP32CAM // No PSRAM
//#define CAMERA_MODEL_M5STACK_UNITCAM // No PSRAM
#define CAMERA_MODEL_AI_THINKER // Has PSRAM
//#define CAMERA_MODEL_TTGO_T_JOURNAL // No PSRAM
//#define CAMERA_MODEL_XIAO_ESP32S3 // Has PSRAM
// ** Espressif Internal Boards **
//#define CAMERA_MODEL_ESP32_CAM_BOARD
//#define CAMERA_MODEL_ESP32S2_CAM_BOARD
//#define CAMERA_MODEL_ESP32S3_CAM_LCD
//#define CAMERA_MODEL_DFRobot_FireBeetle2_ESP32S3 // Has PSRAM
//#define CAMERA_MODEL_DFRobot_Romeo_ESP32S3 // Has PSRAM
#include "camera_pins.h"

// ===========================
// Enter your WiFi credentials
// ===========================
const char* ssid = "kazu";
const char* password = "kazuha13";

void startCameraServer();
void setupLedFlash(int pin);

const int servoPin = 14;
const int buttonPin = 13;
```

```
Servo myservo;
ezButton mybutton(buttonPin);

int currentAngle = 90;

void ServoFaceTracking(int position_x, int WIDE) {
  int margin = 30;
  int CENTRE = WIDE / 2;

  if (position_x < (CENTRE - margin)) {
    if (currentAngle < 180) {
      currentAngle += 10;
      myservo.write(currentAngle);
    }
  }
  else if (position_x > (CENTRE + margin)) {
    if (currentAngle > 0) {
      currentAngle -= 10;
      myservo.write(currentAngle);
    }
  }
}

void setup() {


  myservo.attach(servoPin);
  myservo.write(currentAngle);

  mybutton.setDebounceTime(50);

  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Serial.println();

  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
```

```
  config.pin_reset = RESET_GPIO_NUM;
  config.xclk_freq_hz = 20000000;
  config.frame_size = FRAMESIZE_UXGA;
  config.pixel_format = PIXFORMAT_JPEG; // for streaming
  config.pixel_format = PIXFORMAT_RGB565; // for face
detection/recognition //comment
  config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
  config.fb_location = CAMERA_FB_IN_PSRAM;
  config.jpeg_quality = 12;
  config.fb_count = 1;

  // if PSRAM IC present, init with UXGA resolution and higher JPEG
quality
  //                     for larger pre-allocated frame buffer.
  if(config.pixel_format == PIXFORMAT_JPEG){
    if(psramFound()){
      config.jpeg_quality = 10;
      config.fb_count = 2;
      config.grab_mode = CAMERA_GRAB_LATEST;
    } else {
      // Limit the frame size when PSRAM is not available
      config.frame_size = FRAMESIZE_SVGA;
      config.fb_location = CAMERA_FB_IN_DRAM;
    }
  } else {
    // Best option for face detection/recognition
    config.frame_size = FRAMESIZE_240X240;
#if CONFIG_IDF_TARGET_ESP32S3
    config.fb_count = 2;
#endif
  }

#if defined(CAMERA_MODEL_ESP_EYE)
  pinMode(13, INPUT_PULLUP);
  pinMode(14, INPUT_PULLUP);
#endif

  // camera init
  esp_err_t err = esp_camera_init(&config);
  if (err != ESP_OK) {
    Serial.printf("Camera init failed with error 0x%x", err);
    return;
  }

  sensor_t * s = esp_camera_sensor_get();
  // initial sensors are flipped vertically and colors are a bit
saturated
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1); // flip it back
    s->set_brightness(s, 1); // up the brightness just a bit
    s->set_saturation(s, -2); // lower the saturation
  }
  // drop down frame size for higher initial frame rate
```

```
  if(config.pixel_format == PIXFORMAT_JPEG){
    s->set_framesize(s, FRAMESIZE_QVGA);
  }

#if defined(CAMERA_MODEL_M5STACK_WIDE) ||
defined(CAMERA_MODEL_M5STACK_ESP32CAM)
  s->set_vflip(s, 1);
  s->set_hmirror(s, 1);
#endif

#if defined(CAMERA_MODEL_ESP32S3_EYE)
  s->set_vflip(s, 1);
#endif

// Setup LED FLash if LED pin is defined in camera_pins.h
#if defined(LED_GPIO_NUM)
  setupLedFlash(LED_GPIO_NUM);
#endif

  WiFi.begin(ssid, password);
  WiFi.setSleep(false);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("");
  Serial.println("WiFi connected");

  startCameraServer();

  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  Serial.println("' to connect");
}

void loop() {
mybutton.loop();


 if (mybutton.getState() == LOW) {
    if (currentAngle < 180) {
      currentAngle += 1;
      myservo.write(currentAngle);
      Serial.println(currentAngle);
      delay(10);
    }
 }
}
```

5.0 DATA COLLECTION

5.1 Task 1

This table records the system's responsiveness to physical user input through the GPIO pin.

Pushbutton State	Servo Motion	Observations
Pressed	Rotating	Servo moves while the button is held.
Released	Stationary	Servo stops immediately.
Pressed	Rotating	Servo moves while the button is held.

5.2 Task 2

This table tracks the integration between the ESP32-CAM camera's vision processing and the servo's tracking response.

Face Location	Servo Motion
Left	Rotates 180° to the left
Center	Stationary
Right	Rotates 180° to the right

6.0 DATA ANALYSIS

6.1 Task 1

The data from Task 1 demonstrates a direct, low-latency relationship between manual user input and the mechanical response of the surveillance system. By utilizing a pushbutton connected to a GPIO pin, the system transitioned from an idle state to an active panning state only when the "Pressed" condition was met. The observation that the servo stopped immediately upon release indicates successful implementation of the pull-up resistor logic, which effectively filtered out signal noise and ensured the ESP32-CAM only executed movement commands during deliberate physical engagement. This confirms the system's reliability as a manually steerable surveillance platform.

6.2 Task 2

The analysis of Task 2 highlights the successful integration of computer vision with mechatronic actuation. The data shows that the ESP32-CAM was able to process face location coordinates in real-time and translate them into specific rotational commands for the servo motor. When the face was detected on the left or right, the servo responded with a full 180° rotation to reposition the camera, while a "Center" detection resulted in a stationary state to maintain focus on the subject. This behavior validates the use of a coordinate-based tracking system, proving that the surveillance camera can autonomously adjust its field of view to follow a detected target across a wide area.

7.0 RESULTS

7.1 Task 1

In this experiment, we can see that when the push button is pressed, the servo motor will move. After the push button is released, the angle of servo motor back to 0. This feature is included so we do not need to reset the angle of the servo motor every time we want to use it.

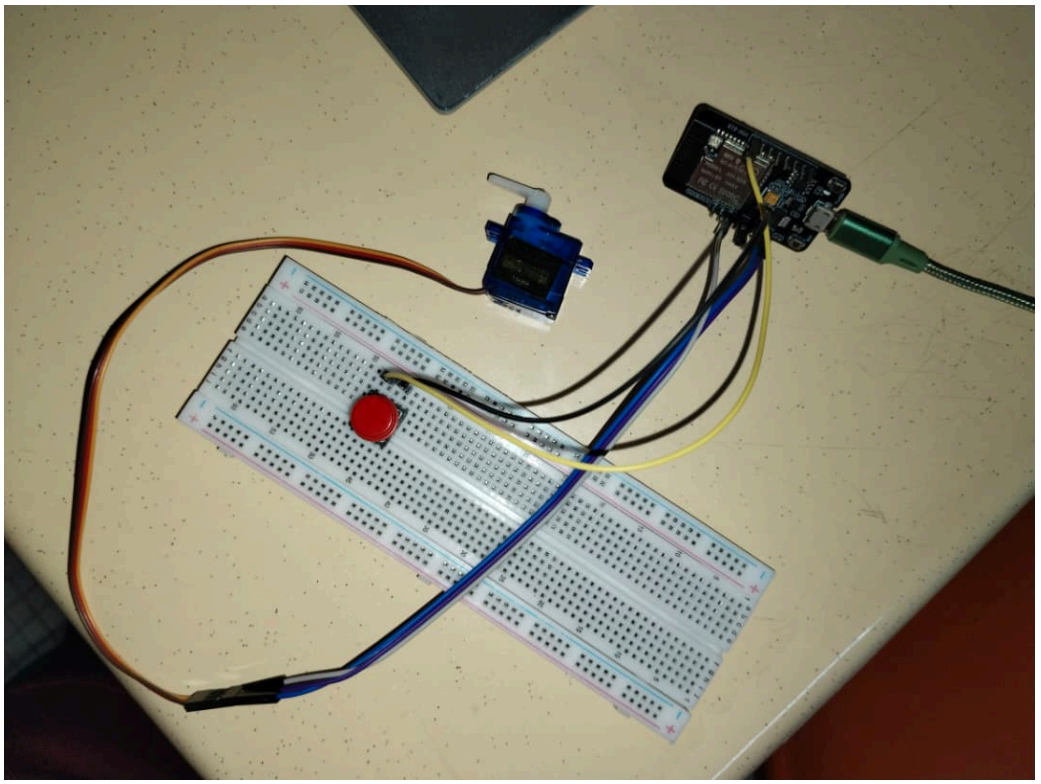


Diagram 7.1.1 show the installation of the circuit

7.2 Task 2

When wifi is connected with the Esp32-Cam we can open the website using our phone or laptop to see what the camera records. After that, if we point the camera to our face, the camera will detect and a box will appear marking our face. If our face is at the centre of the camera then the servo will stay at 90 degrees. When we move our face to the left, the servo motor also will move the left following our face direction. The same things happen when we move our face to the right.

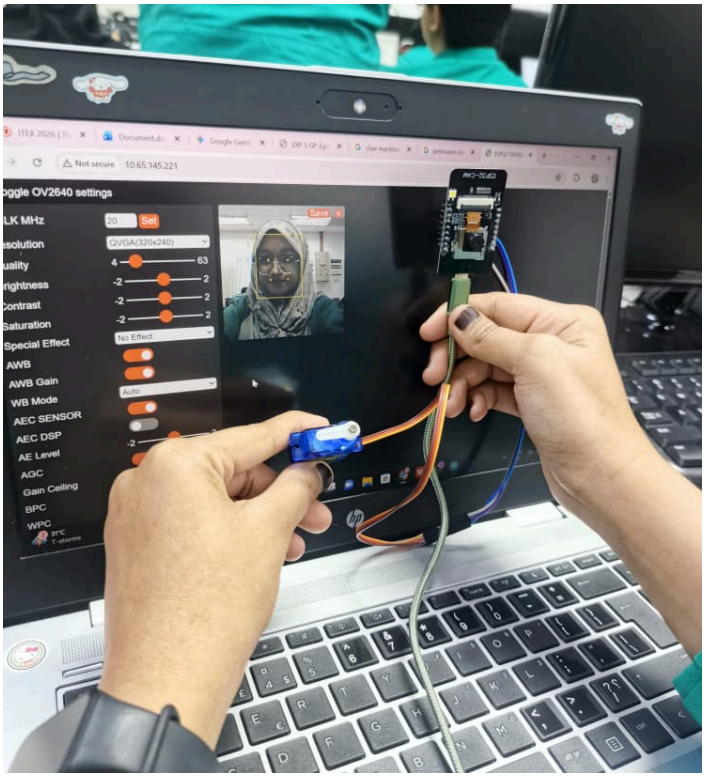


Diagram 7.2.1 show when the camera detect a face

8.0 DISCUSSION

8.1 Task 1

In this experiment, the system successfully modified to transition from continuous moving servo to a manual control servo using a push button. By utilizing the ezButton library, the system effectively handled bouncing signals from the mechanical switch, ensuring that the servo motor only incremented its position when a stable LOW state was detected.

8.2 Task 2

The system was upgraded from manual control of servo motors to automatic tracking system using the Esp32-CAM built in face detection capabilities. To implement autonomous tracking, the standard “CameraWebServer” example was modified within the “app_httpd” file. This modification is crucial because it allows the system to intercept the metadata generated by the face detection algorithm. By accessing the “results.front()” object, the system specifically targets the first face detected in the frame, ensuring a singular point of focus for the actuator. The logic “(myface.box[0] + myface.box[2]) / 2” is used to calculate the horizontal centroid (center point) of the detected face's bounding box. This value, “face_x”, provides a precise pixel location which is then compared against the total frame width “(rfb.width)”. After that, the logic utilizes the X-coordinate of the detected face relative to the frame width “(WIDE)”. By defining a “CENTRE” point and a “margin” (30 pixels), the system creates a "dead zone" that prevents the servo from oscillating when the face is relatively centered. Furthermore, this system is a closed-loop feedback system, the ESP32-CAM acts as the visual sensor (Input), the centroid calculation serves as the error-detection logic (Controller), and the servo motor acts as the final control element (Actuator).

9.0 CONCLUSION AND RECOMMENDATION

9.1 Task 1

In conclusion, we can use Esp32 as a microcontroller other than Arduino Uno. The coding in the Esp32 and the library is a little different from the Arduino Uno. But the Esp32 only can power up to 3.3V while Arduino Uno can power up to 5V and have more digital pins. We can integrate both Arduino Uno and Esp32 to make a better functioning system. My recommendation for this experiment is to check the wiring carefully as Esp32 is easy to get 'fried' compared to Arduino.

9.2 Task 2

To conclude, the Esp32 device is a great microcontroller as it can connect to wifi and we can control it via our phone or laptop. The project also successfully demonstrates the integration of AI tasks such as face detection and physical actuator. These experiments also show how AI can make manual controls of servo become automatic using face detection. For recommendation, we can change the movements of the servo motor. The motor only moves with the increment of 2. If the faces move very fast, the servo can not catch up with the face.

ACKNOWLEDGEMENT

We would like to sincerely express our gratitudes to our instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal and the lab technician, for the help given during the task. The knowledge and suggestions are able to guide us throughout the progress of the task. We also would like to thank our classmates for their support and help which really give positive impacts to the progression of our task.

STUDENTS' DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been untaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed

unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature:  Read [/]

Name: NOR EFFENDI BIN ANUAR @ ANWAR Understand [/]

Matric Number: 2310519 Agree [/]

Signature:  Read [/]

Name: NUR AINA SAFFIYA BINTI MOHD TARMIZI Understand [/]

Matric Number: 2411644 Agree [/]

Signature:  Read [/]

Name: NUR ATIKAH BINTI KAMARUL BAHARIN Understand [/]

Matric Number: 2319846 Agree [/]